Lab 06: Source Control Management Using Git: Student Lab Handout

DevOps, BS Program. Part A: Notes. Part B: Lab activities. Complete every activity in order and show the checkpoints to your instructor.

PART A: NOTES

A1. Objective and Outcomes

Objective: to perform version control on websites and software using Git with the `push` and `pull` commands.

After this lab you will be able to:

- Explain the different version control strategies.
- Connect a local repository to a GitHub repository.
- Upload changes with `git push` and download changes with `git pull`.
- See remote changes without merging them using `git fetch`.
- View, compare, and undo versions of a project, and mark a release with a tag.

Before you start: Git must be installed, and you must know `git init`, `git status`, `git add`, `git commit`, `git log`, and `git branch` from Lab 05.

A2. What is Source Control Management?

Source Control Management (SCM), also called a **Version Control System (VCS)**, tracks changes to files over time. For a website (`index.html`, CSS, images) or a software project, every edit creates a new *version*. With version control you can:

- **See history:** who changed what, when, and why.
- **Roll back:** return to any earlier working version.
- **Collaborate:** many people work on one project without overwriting each other.
- **Back up:** the project is stored on several machines and on a server.
- **Experiment:** try new ideas on a branch without breaking stable code.

A3. Version Control Strategies

Feature	Local	Centralized (CVS, SVN, Perforce)	Distributed (Git, Mercurial)
Where is the history?	One computer	One central server	Every developer's copy
Works offline?	Yes	Limited	Yes
Collaboration	No	Yes	Yes
Single point of failure	Yes	Yes (the server)	No
Branching	Manual folder copies	Heavy	Cheap and fast

Git is distributed: every clone is a full copy of the repository with its entire history, so any clone can restore a lost server.

Common team workflows (same commands, different rules):

- **Centralized workflow:** everyone commits to one `main` branch (small teams).
- **Feature branch workflow:** every feature or fix gets its own branch and is merged into `main` through a pull request.
- **GitHub Flow:** `main` is always deployable and work happens on short-lived branches.
- **Git Flow:** long-lived branches such as `develop`, `release`, and `hotfix`.
- **Forking workflow:** contributors fork a project, change their copy, and send a pull request (common in open source).

A4. Git vs GitHub

Git	GitHub
A tool installed on your computer	A website that hosts Git repositories
Works locally and offline	Works online
Commands: `add`, `commit`, `log`	Features: pull requests, issues, forks, Actions

A5. The Four Areas and the Push/Pull Arrows

```
Working Directory --git add--> Staging Area --git commit--> Local Repo --git
push--> Remote Repo (GitHub)
                                                                    ^
|
                                                                    +---- git fetch
/ git pull ----+
```

- **Push** sends commits from your local repository to the remote repository.
- **Pull** brings changes from the remote into your local repository **and merges** them.
- **Fetch** brings the remote changes into your local repository but **does not merge** them.

> **Important:** you must **commit first**. Push only uploads commits that already exist locally. If you did not commit, there is nothing to push.

A6. GitHub Account and Repository

1. Go to **github.com**, click **Sign up**, and create an account.
2. **Verify your email address** (GitHub sends you a code or link).
3. Click **+**, then **New repository**. Enter a name and choose **Public**.
4. For a project you will push from your computer, **do not tick "Add a README"**. Keep the repository empty.
5. Click **Create repository**, then copy the URL:

```
https://github.com/<username>/<repository>.git
```

A7. Authentication (read this before pushing)

GitHub **does not accept your account password** for command-line access. When Git for Windows shows a message like `Logon failed, use ctrl+c to cancel basic credential prompt`, sign in using one of these:

- A **browser sign-in popup** (Git Credential Manager, installed with Git for Windows), or
- A **Personal Access Token (PAT)** typed in place of the password. To create one: GitHub, **Settings**, **Developer settings**, **Personal access tokens**, generate a token with the **repo** scope, and **copy it immediately** (it is shown only once), or

- **SSH keys**.

Keep your token private, like a password.

A8. Remotes

A **remote** is a short name for a repository URL. The usual name is `origin`, but any name works (the lab manual uses `example`).

```
git remote add origin <url>      # create a remote
git remote -v                    # list remotes and URLs
git remote rename old new        # rename a remote
git remote remove <name>         # delete a remote
git remote set-url origin <url>  # fix a wrong URL
```

A9. Push

```
git push -u <remote> <branch>
```

- `-u` (`--set-upstream`) links your local branch to the remote branch. After that you can just type `git push`.
- Typical output lines and their meaning:

Output	Meaning
`Enumerating objects` / `Counting objects`	Git works out what the server is missing
`Writing objects: 100%`	Data is being uploaded
`remote: Create a pull request for ...`	GitHub received a new branch and offers a pull request link
`* [new branch] master -> master`	A new remote branch was created
`Branch 'master' set up to track remote branch`	The `-u` link was created

- GitHub's default branch is now **`main`**. If your local branch is `master`, GitHub will show **two branches**. To rename yours: `git branch -M main`.

A10. Pull

```
git pull                            # in a repository that already has a remote
git pull <url>                      # download from a URL into the current
repository
git pull <remote> <branch>
```

- **Pull = fetch + merge.**
- `FETCH_HEAD` in the output is a temporary pointer to what was just downloaded.
- **Pull vs clone:** `git clone <url>` creates the folder, downloads everything, and saves the remote as `origin`. Using `git init` and then `git pull <url>` does the same download in two steps but **does not save a remote**.
- Rule of thumb: **clone once, pull many times.** Pull before you start work and before you push.

A11. Fetch

```
git fetch <remote>
git log --oneline <remote>/<branch>
```

- Fetch downloads new commits but **does not change your working files**.
- `example/master` is a **remote-tracking branch**: your local record of where `master` on the remote `example` currently is.
- To bring the changes into your branch afterwards: `git merge <remote>/<branch>` (or use `git pull`).

`git fetch`	`git pull`
Downloads only	Downloads **and** merges
Does not change your files	Changes your files
Safe: review first	May cause a merge conflict

Analogy: fetch is checking the mailbox; pull is bringing the mail in and opening it.

Useful commands after a fetch:

```
git branch -r                          # list remote-tracking branches
git diff main origin/main              # see what changed
git log main..origin/main              # commits you do not have yet
```

A12. Seeing and Undoing Versions

```
git log --oneline                 # list versions
git diff HEAD~1 HEAD              # what changed in the last commit
git show <commit-id>              # details of one commit
git revert <commit-id>            # SAFE undo: adds a new commit that cancels
an old one
git restore <file>                # discard UNCOMMITTED edits to a file
git tag -a v1.0 -m "First release"
git push <remote> v1.0            # tags are not pushed automatically
```

- `git reset --hard <id>` destroys later work. **Never use it on commits you have already pushed.**
- **Semantic versioning:** `MAJOR.MINOR.PATCH` (for example `2.4.1`): MAJOR = breaking change, MINOR = new feature, PATCH = bug fix.

A13. Common Errors and Fixes

Message	Cause	Fix
`Authentication failed` / `Logon failed`	Account password used	Use the browser sign-in or a Personal Access Token
`remote origin already exists`	Remote added twice	`git remote set-url origin <url>`
`src refspec main does not match any`	Branch is `master`, or there are no commits	Commit first, or run `git branch -M main`
`Updates were rejected ... fetch first`	Remote has commits you lack	`git pull`, then push again
`refusing to merge unrelated histories`	Remote and local repos have separate histories	`git pull <remote> <branch> --allow-unrelated-histories`

Message	Cause	Fix
Merge conflict	Same lines edited in two places	Edit the markers `<<<<<<<`, `=======`, `>>>>>>>`, then `git add` and `git commit`
`fatal: not a git repository`	Not inside a repo folder	`cd` into it, or `git init`

PART B: LAB ACTIVITIES

Rules: type every command yourself, read each output carefully, and write your observations in the boxes. Replace `<username>` and `<repo>` with your own values. At each **CHECKPOINT** call the instructor and show your screen.

Activity 0: Setup Check

Goal: make sure Git and your identity are ready.

1. Open **Git Bash** (Windows: right-click, *Git Bash Here*) or the Ubuntu terminal.
2. Run:

```
git --version
git config --global user.name
git config --global user.email
```

3. If name or email is empty, set them:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

4. Make sure you have a **verified GitHub account** and a **Personal Access Token** (see A7).

Question	Your answer
Git version installed	
GitHub username	

CHECKPOINT 0: show the Git version and your GitHub username.

Activity 1: Create a Local Website Repository

Goal: build a small project and commit it locally.

```
mkdir my-website
cd my-website
git init
```

Create two files:

```
echo "<html><head><title>Examples</title></head><body><h1>This is
heading</h1></body></html>" > index.html
echo "Hello all" > readme.txt
git status
git add .
git commit -m "First Commit"
git log --oneline
```

Question	Your answer
What colour/state did `git status` show for the new files before `git add`?	
What is the short ID (hash) of your first commit?	
Which branch are you on (`master` or `main`)?	

Rename the branch to `main` so it matches GitHub:

```
git branch -M main
```

CHECKPOINT 1: show `git log --oneline` with one commit.

Activity 2: Push to GitHub

Goal: upload your local repository to GitHub.

1. On GitHub create a **new empty repository** named `my-website` (Public, **no README**). Copy its URL.
2. In your terminal (inside `my-website`):

```
git remote add origin https://github.com/<username>/my-website.git
git remote -v
git push -u origin main
```

3. Sign in with the browser popup or paste your Personal Access Token when asked.
4. Refresh the GitHub page and check that `index.html` and `readme.txt` appear.

Question	Your answer
What does `git remote -v` show?	
Copy the output line that starts with `* [new branch]`	
What does the `-u` flag do?	
How many branches does GitHub show?	

5. Make a small change and push again:

```
echo "<p>Welcome to my site</p>" >> index.html
git add index.html
git commit -m "Add welcome text"
git push
```

Notice that the second push needs **no remote or branch name**. Explain why:

_____

CHECKPOINT 2: show your repository on GitHub with two commits.

Activity 3: Pull a Project from GitHub

Goal: download a public project into a new folder using `git pull`.

1. Your instructor will give you a public repository URL (or use any small public repository). Write it here: _____
2. Run:

```
cd ..
mkdir downloaded-project
cd downloaded-project
git init
git pull <public-repository-url>
```

3. Inspect what arrived:

```
ls
git log --oneline
git status
git remote -v
```

Question	Your answer
How many objects were received (see "Receiving objects")?	
What does `FETCH_HEAD` in the output mean?	
Did `git remote -v` list a remote? Why or why not?	

4. Now download the **same** project with `git clone` into another folder:

```
cd ..
git clone <public-repository-url> cloned-project
cd cloned-project
git remote -v
```

Question	Your answer
What is different about `git remote -v` here?	
Write one difference between clone and init plus pull	

CHECKPOINT 3: show both folders and the `git remote -v` output of each.

Activity 4: Fetch and Merge Remote Changes

Goal: see a remote change with `fetch` before merging it.

1. Go back into your project: `cd ../my-website` (adjust the path if needed).
2. **On GitHub** (in the browser) open `index.html`, click the pencil icon, change the heading text, and commit it with the message `Update index.html`.
3. In your terminal run:

```
git log --oneline                    # note the top commit
git fetch origin
git log --oneline origin/main        # the new commit appears here
git status
cat index.html                       # has your file changed?
```

Question	Your answer
Does `git log --oneline` (local) show the new commit?	
Does `git log --oneline origin/main` show it?	
Did `index.html` on your computer change after the fetch?	
What does `origin/main` mean?	

4. Compare, then bring in the change:

```
git diff main origin/main
git merge origin/main
cat index.html
```

5. Edit another line **on GitHub**, then this time use pull:

```
git pull
```

Question	Your answer
Which command changed your files without asking: fetch or pull?	

Question	Your answer
Explain pull in one line using fetch and merge	

CHECKPOINT 4: show `git log --oneline origin/main` before and after the merge.

Activity 5: Version Control of a Website

Goal: create versions, find a mistake, and undo it safely.

1. Create three versions:

```
echo "<h2>About us</h2>" >> index.html
git commit -am "v2: add about section"

echo "<p>broken tag" >> index.html
git commit -am "v3: add footer"
```

(`-am` stages and commits files that Git already tracks.)

2. Inspect the history:

```
git log --oneline
git diff HEAD~1 HEAD
git show HEAD
```

3. Version 3 contains a mistake. **Undo it safely:**

```
git revert HEAD
git log --oneline
```

4. Try discarding an **uncommitted** edit:

```
echo "temporary text" >> readme.txt
git status
git restore readme.txt
git status
```

5. Mark a release and push everything:

```
git tag -a v1.0 -m "First release"
git push origin main
git push origin v1.0
```

Open the **Tags/Releases** area on GitHub and confirm `v1.0` is there.

Question	Your answer
What did `git diff HEAD~1 HEAD` show?	
How many commits are in your log after `git revert`? Why not fewer?	
Why is `git revert` safer than `git reset --hard` for pushed commits?	
What does a tag give us?	

CHECKPOINT 5: show the log with the revert commit and the tag on GitHub.

Activity 6: Work on a Branch and Open a Pull Request

Goal: push a feature branch and merge it using GitHub.

```
git switch -c contact-page
echo "<p>Contact: student@example.com</p>" > contact.html
git add contact.html
git commit -m "Add contact page"
git push -u origin contact-page
```

1. Read the output line that begins with `remote: Create a pull request for`.
2. On GitHub, click **Compare & pull request**, write a short description, and click **Create pull request**, then **Merge pull request**.
3. Update your local `main`:

```
git switch main
git pull
git log --oneline --graph
ls
```

Question	Your answer
Is `contact.html` in your `main` branch after the pull?	
What is the purpose of a pull request?	

CHECKPOINT 6: show the merged pull request and `git log --oneline --graph`.

Activity 7: Fix These Problems (Troubleshooting)

Goal: recognize and solve common errors.

Problem A: rejected push.

1. On GitHub, edit `readme.txt` and commit the edit.
2. Locally, edit `readme.txt` on a **different line** and commit.
3. Run `git push`. Copy the error message here: _____
4. Fix it with `git pull`, then `git push` again.

Problem B: wrong remote URL.

```
git remote add backup https://github.com/<username>/wrong-name.git
git push backup main
```

Copy the error message here: _____

Fix the URL and list the remotes:

```
git remote set-url backup https://github.com/<username>/my-website.git
git remote -v
```

Problem C: nothing to push. Run `git push` right after a successful push. What does Git print? _____ Why?

CHECKPOINT 7: explain Problems A and B to your instructor.

PART C: GRADED LAB TASK

Lab Task 1: Create a new Git repository and upload it to GitHub using push

Without looking at the steps above, do the following in a **new** folder named `lab6-task1`:

1. Initialize a Git repository.
2. Create `index.html` (with a heading and a paragraph) and `readme.txt`.
3. Commit with a meaningful message.
4. Create an **empty** repository named `lab6-task1` on GitHub.
5. Connect it as a remote and push to it, setting the upstream.
6. Show the files on GitHub.
7. Make one more change on your computer, commit, and push using only `git push`.
8. Explain to the instructor: the difference between push and pull, the difference between fetch and pull, and what `-u` does.

Extension (optional): edit a file on GitHub and bring the change into your computer with `git fetch` and `git pull`.

PART D: REVIEW QUESTIONS (Write in Your Notebook)

1. What is source control management, and why do websites and software projects need it?
2. Compare local, centralized, and distributed version control. Why is Git called distributed?
3. What is the difference between Git and GitHub?
4. What is a remote? Is `origin` a required name?
5. What does the `-u` flag do in `git push -u origin main`?
6. What is the difference between `git fetch` and `git pull`?
7. What does `origin/main` mean in a `git log` output?
8. Why does GitHub ask for a Personal Access Token and not a password?
9. What is the difference between `git revert` and `git reset --hard`?

10. What is the difference between `git clone` and `git init` followed by `git pull`?

SUBMISSION CHECKLIST

- [ ] All checkpoints (0 to 7) signed by the instructor
- [ ] Tables in Part B filled in
- [ ] Repository `my-website` visible on GitHub with a tag `v1.0`
- [ ] Repository `lab6-task1` visible on GitHub
- [ ] Review questions answered in the notebook